

DOCUMENTO TÉCNICO DE DISEÑO

IMPLEMENTACIÓN TÉCNICA COMPLETA EN POSTGRESQL Y MONGODB - ETAPA DOS

DISEÑO Y OPTIMIZACIÓN DE BASES DE DATOS

EQUIPO E21

NESTOR ALEJANDRO RODRIGUEZ BENAVIDES - NESTORROBE@UNISABANA.EDU.CO

CARLOS DANIEL SANDOVAL - CARLOSSANDPAR@UNISABANA.EDU.CO

PETER ALEXANDER PALACIOS GARNICA - PETERPAGA@UNISABANA.EDU.CO

JUAN GUILLERMO OSSA SÁNCHEZ - JUANOSSA@UNISABANA.EDU.CO

MAESTRÍA EN ARQUITECTURA DE SOFTWARE

DOCENTE: MIGUEL ALFONSO VARELA

16 DE JUNIO DE 2026

Índice

Índice.....	2
1. Resumen ejecutivo.....	3
2. Alcance y continuidad con Ecommify.....	3
3. Implementación PostgreSQL en Supabase.....	4
4. Optimización PostgreSQL.....	5
5. Implementación MongoDB en Atlas.....	6
6. Optimización MongoDB.....	7
7. Evidencias cuantitativas consolidadas.....	9
8. Sincronización entre PostgreSQL y MongoDB.....	9
9. Sharding y replica sets.....	10
10. Conclusiones.....	10
11. Referencias.....	10
Anexos.....	11
Anexo A. Evidencias PostgreSQL.....	11
Anexo B. Evidencias MongoDB.....	11

1. Resumen ejecutivo

En esta unidad se consolida la implementación técnica completa de Ecommify sobre una arquitectura híbrida PostgreSQL/Supabase + MongoDB Atlas. PostgreSQL conserva el rol de núcleo transaccional para pedidos, pagos, inventario, clientes, vendedores y datos maestros; MongoDB complementa la solución con catálogo enriquecido, reseñas, búsquedas flexibles, agregaciones y patrones documentales.

La entrega integra scripts, validaciones, evidencias de rendimiento. En PostgreSQL se confirma el esquema core, sus tablas principales, constraints, extensiones, índices, particionamiento de core.orders y mediciones EXPLAIN heredadas de la Unidad 4. En MongoDB se documentan product_catalog, product_reviews, product_review_buckets, validación con JSON Schema, índices compuestos/parciales/texto y pipeline de agregación optimizado.

Los resultados muestran mejoras cuantitativas relevantes en PostgreSQL para Q08, Q09 y Q02; en MongoDB, la consulta Q04 cambió de COLLSCAN a IXSCAN mediante índice de texto, Q07 redujo tiempo de ejecución, y el Bucket Pattern agrupó 102.172 reseñas fuente en 33.038 documentos bucket para facilitar consultas analíticas.

Tabla 2. Resultados ejecutivos destacados.

Frente	Resultado	Dato destacado	Lectura técnica
PostgreSQL	Validación final del esquema core	8/8 validaciones OK	Supabase conserva tablas, constraints, índices, particionamiento y consulta de control con EXPLAIN.
PostgreSQL	Mejores reducciones de tiempo	Q08 80,29%; Q09 74,55%; Q02 66,70%	Evidencia tomada del notebook U4 Etapa 2 y referenciada en el ZIP mediante manifiesto.
MongoDB	Cambio de plan de búsqueda textual	Q04: COLLSCAN -> IXSCAN	El índice idx_text_product_search quedó como soporte para búsqueda full-text.
MongoDB	Bucket Pattern	102.172 reseñas -> 33.038 buckets	Agrupación por product_id, bucket_period y bucket_sequence.
Repositorio	Estructura final	64 artefactos inventariados	README raíz, README por motor, evidencias, scripts, notebooks y documentación.

2. Alcance y continuidad con Ecommify

La implementación no reemplaza el diseño conceptual y lógico ya definido para Ecommify. Lo consolida con artefactos ejecutables, evidencias de optimización y documentación de repositorio. El enfoque sigue siendo transaccional-analítico: las operaciones críticas permanecen en PostgreSQL y las vistas documentales o analíticas se materializan en MongoDB Atlas.

Tabla 3. Componentes de la arquitectura y rol en la implementación.

Componente	Rol	Artefactos	Decisión técnica
PostgreSQL/Supabase	Fuente transaccional principal	core.orders, core.order_items, core.payments, core.inventory	Consistencia, integridad referencial y particionamiento.
MongoDB Atlas	Módulo documental y analítico	product_catalog, product_reviews, product_review_buckets	Flexibilidad documental, búsqueda textual, agregaciones y patrones de diseño.
Google Colab	Ejecución y validación	Notebooks de carga, optimización y evidencia	Permite repetir pruebas sin instalar herramientas locales.
GitHub	Repositorio central	README, scripts, notebooks, evidencias, docs	Trazabilidad, reproducibilidad y entrega evaluativa.

3. Implementación PostgreSQL en Supabase

La implementación relacional está organizada alrededor del esquema core. Allí se ubican las tablas principales del dominio Ecommify: geolocation, product_categories, customers, sellers, products, orders, order_items, payments, reviews_ref e inventory. Esta separación favorece claridad de dominio, control transaccional e integración con las consultas críticas.

Tabla 4. Tablas principales verificadas en PostgreSQL/Supabase.

Grupo	Tablas	Propósito
Maestras y referencia	core.geolocation, core.product_categories, core.customers, core.sellers, core.products	Soportan normalización, integridad y reutilización de atributos comunes.
Transaccionales	core.orders, core.order_items, core.payments	Concentran pedidos, ítems y pagos; requieren ACID y constraints.
Complementarias	core.reviews_ref, core.inventory	Reseñas referenciales e inventario derivado para operación y análisis.

Tabla 5. Inventario de scripts PostgreSQL documentados en el repositorio.

Categoría	Archivo	Ruta	Tamaño bytes
schema	00_extensions.sql	postgresql/schema/00_extensions.sql	371
schema	01_schemas.sql	postgresql/schema/01_schemas.sql	312
schema	02_types_domains.sql	postgresql/schema/02_types_domains.sql	1008
schema	03_tables_core.sql	postgresql/schema/03_tables_core.sql	1863
schema	04_tables_transactions.sql	postgresql/schema/04_tables_transactions.sql	2846
schema	05_indexes.sql	postgresql/schema/05_indexes.sql	1234
schema	06_triggers_updated_at.sql	postgresql/schema/06_triggers_updated_at.sql	984
schema	07_partitioning_orders.sql	postgresql/schema/07_partitioning_orders.sql	673
schema	08_materialized_views.sql	postgresql/schema/08_materialized_views.sql	1790
queries	11_validation_queries.sql	postgresql/queries/11_validation_queries.sql	719
queries	12_monitoring_queries.sql	postgresql/queries/12_monitoring_queries.sql	773
seed_data	10_seed_data.sql	postgresql/seed_data/10_seed_data.sql	657

Tabla 6. Validación final PostgreSQL/Supabase.

Validación	Estado	Evidencia	Notas
schema_core_exists	OK	Consulta sobre information_schema.schemata	El esquema core existe en Supabase PostgreSQL.
core_tables_exist	OK	Consulta sobre pg_tables	Las tablas principales del modelo Ecommify existen en core.
row_estimates_available	OK	Consulta sobre pg_class y pg_namespace	Las tablas tienen conteos estimados disponibles.
extensions_checked	OK	Consulta sobre pg_extension	Se validaron extensiones relevantes para el modelo.
constraints_checked	OK	Consulta sobre information_schema.table_constraints	Se validaron primary keys foreign keys check constraints y otros constraints.
orders_partitioning_checked	OK	Consulta sobre pg_partitioned_table	La tabla core.orders aparece particionada por purchase_date.
indexes_inventory_checked	OK	Consulta sobre pg_indexes	Se validó inventario de índices del esquema core.
explain_control_query_executed	OK	EXPLAIN ANALYZE BUFFERS	La consulta crítica de control ejecutó correctamente.

Los scripts esperados cubren extensiones, esquemas, dominios/tipos, tablas core, tablas transaccionales, índices, triggers updated_at, particionamiento de orders y vistas materializadas.

4. Optimización PostgreSQL

La optimización PostgreSQL se apoya en el trabajo ejecutado en la Unidad 4 Etapa 2. Allí se midieron nueve consultas críticas con EXPLAIN (ANALYZE, BUFFERS), se diagnosticaron planes, se aplicaron reescrituras SQL, se crearon índices especializados y se volvió a medir el comportamiento posterior.

Tabla 7. Consultas críticas PostgreSQL Q01-Q09.

Código	Consulta crítica	Tablas principales	Propósito
Q01	Consulta de pedido por identificador	orders, customers	Detalle puntual de una orden.
Q02	Historial de pedidos por cliente	orders, customers	Pedidos de un cliente ordenados por fecha.
Q03	Pedidos por estado y rango de fechas	orders, customers	Órdenes delivered en un periodo.
Q04	Ventas mensuales por categoría	orders, order_items, products, product_categories	Agregación de ventas por mes y categoría.
Q05	Ventas por vendedor	order_items, sellers	Ranking de vendedores por volumen.
Q06	Búsqueda de productos por categoría	products, product_categories	Búsqueda textual/categórica.
Q07	Análisis de pagos por tipo	payments	Métricas por medio de pago.
Q08	Reseñas por calificación	reviews_ref	Análisis de satisfacción por score.
Q09	Inventario por producto y vendedor	inventory	Disponibilidad por producto-vendedor.

Tabla 8. Índices especializados PostgreSQL aplicados/validados.

Índice	Tabla	Tipo	Columnas	Consulta	Justificación
idx_orders_customer_purchase_btree	core.orders	B-tree compuesto	customer_id, purchase_date DESC	Q02	Historial de pedidos por cliente en orden temporal.
idx_orders_delivered_purchase_partial	core.orders	B-tree parcial	purchase_date DESC WHERE order_status = 'delivered'	Q03	Filtra subconjunto delivered y reduce tamaño del índice.
idx_products_product_name_trgm_gin	core.products	GIN + pg_trgm	product_name gin_trgm_ops	Q06	Búsqueda textual con coincidencias parciales.
idx_order_items_purchase_date_brin	core.order_items	BRIN	purchase_date	Q04 Q04_OPT	Índice liviano para rangos temporales de alto volumen.

Tabla 9. Manifiesto de evidencias EXPLAIN antes/después para PostgreSQL.

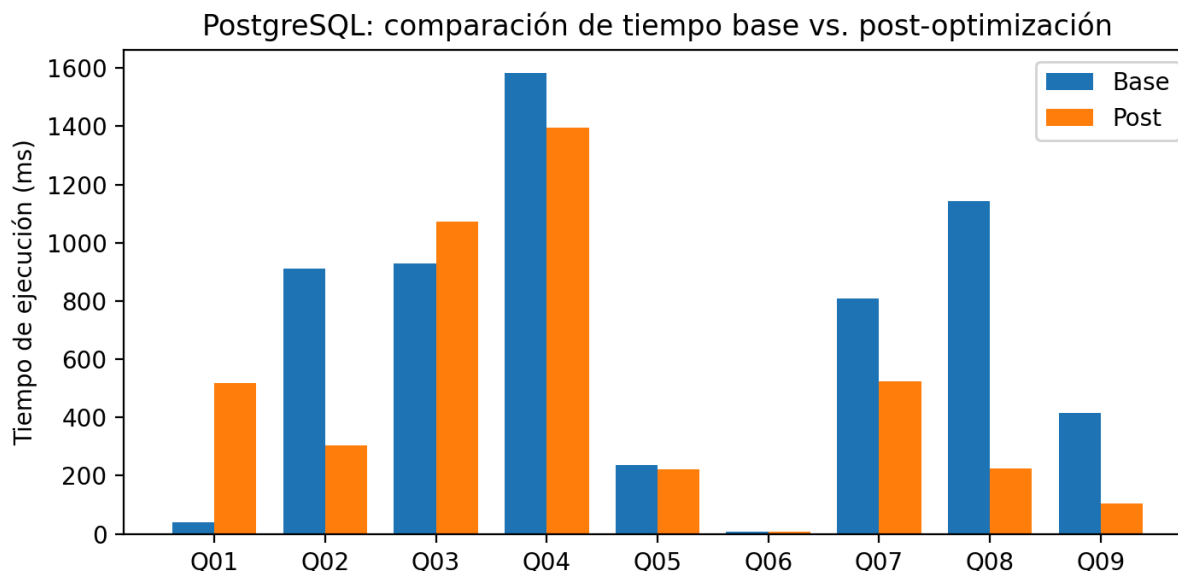
Evidencia	Estado	Descripción	Referencia fuente
baseline_explain	documented	Medición base de Q01-Q09 con EXPLAIN ANALYZE BUFFERS	U4 Etapa 2 Hito 4
post_optimization_explain	documented	Medición posterior después de optimizaciones e índices	U4 Etapa 2 Hito 8
before_after_comparison	documented	Comparación antes/después con reducción de tiempo y bloques leídos cuando aplica	U4 Etapa 2 Hito 8
specialized_indexes	documented	Índices B-tree parcial GIN pg_trgm y BRIN documentados	U4 Etapa 2 Hito 7
partitioning_validation	documented	Validación de particionamiento declarativo de core.orders por purchase_date	U4 Etapa 2 Hito 9

Tabla 10. Comparación PostgreSQL antes/después desde notebook U4 Etapa 2.

Consulta	Nombre	Tiempo base (ms)	Tiempo post (ms)	Reducción (%)	Interpretación
Q01	Consulta de pedido por identificador	39,488	519,345	-1.215,20	Regresión por baja selectividad/caché; se conserva como evidencia de medición.
Q02	Historial de pedidos por cliente	909,747	302,991	66,70	Mejora relevante con índice por cliente y fecha.
Q03	Pedidos por estado y rango de fechas	928,066	1.072,731	-15,59	Plan no mejoró; requiere revisar selectividad y particiones.
Q04	Ventas mensuales por categoría	1.583,998	1.394,929	11,94	Mejora moderada; reducción de lecturas compartidas de 68,51%.
Q05	Ventas por vendedor	237,373	221,161	6,83	Mejora leve por preagregación y soporte de índices.
Q06	Búsqueda de productos por categoría	7,422	6,325	14,78	Mejora leve; consulta ya era rápida.
Q07	Análisis de pagos por tipo	809,671	523,651	35,33	Mejora visible en consulta analítica.

Consulta	Nombre	Tiempo base (ms)	Tiempo post (ms)	Reducción (%)	Interpretación
Q08	Reseñas por calificación	1.143,251	225,299	80,29	Mejor resultado relativo observado.
Q09	Inventario por producto y vendedor	414,951	105,594	74,55	Mejora alta por reescritura y cálculo único de disponibilidad.

Figura 2. PostgreSQL: tiempos base y post-optimización por consulta.



Algunas consultas muestran regresiones o cambios menores por baja selectividad, caché, tamaño de tablas o decisiones válidas del planificador.

5. Implementación MongoDB en Atlas

MongoDB Atlas se implementó como módulo documental y analítico. Las colecciones principales se enfocan en enriquecer el catálogo, referenciar reseñas y construir estructuras para análisis eficiente.

Tabla 11. Colecciones MongoDB implementadas.

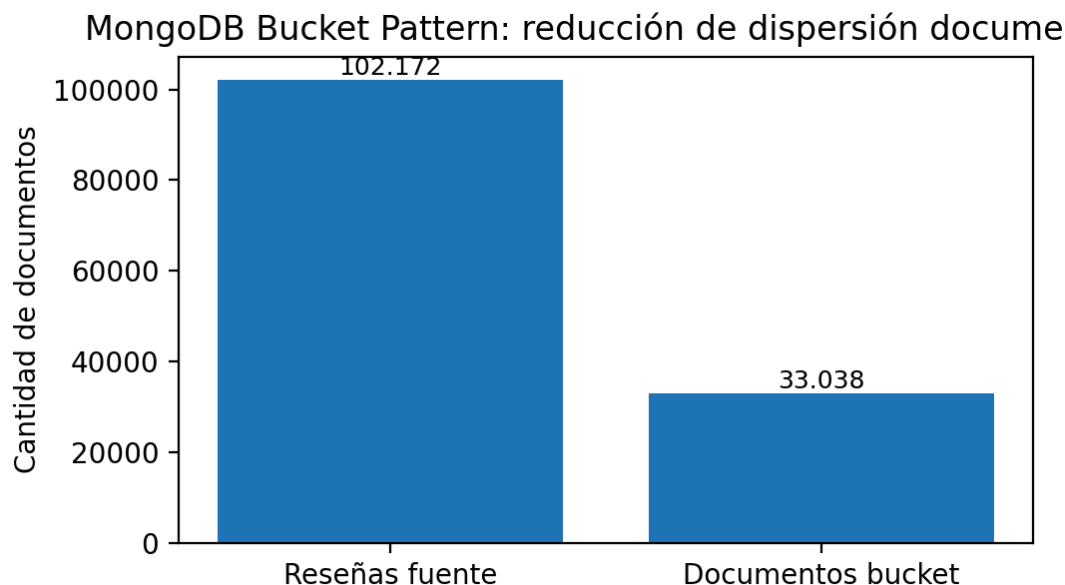
Colección	Rol	Estructura resumida	Patrones aplicados
product_catalog	Catálogo enriquecido	Documento principal con product_id, name, category, price_summary, specifications, seller_summary, rating_summary, sales_summary y search_keywords.	Attribute Pattern, Extended Reference, Computed Pattern
product_reviews	Reseñas referenciadas	Reseñas asociadas por product_id para análisis de satisfacción y consultas por producto.	Referencing + índices por producto/fecha/score
product_review_buckets	Colección derivada	Buckets por product_id, bucket_period y bucket_sequence con métricas precalculadas.	Bucket Pattern

Tabla 12. Evidencia de validación JSON Schema sobre product_catalog.

Prueba	Esperado	Resultado	Mensaje resumido	Fecha
invalid_document_insert	REJECTED	REJECTED	Document failed validation, full error: {'index': 0, 'code': 121, 'errmsg': 'Document failed validation', 'errInfo': {'failingDocumentId': ObjectId('6a31365699c7e82de5167...}}	2026-06-16T11:41:10

Tabla 13. Evidencia de Bucket Pattern en MongoDB.

Patrón	Colección fuente	Colección destino	Agrupación	Límite bucket	Reseñas fuente	Documentos bucket	Propósito
Bucket Pattern	product_reviews	product_review_buckets	product_id + bucket_period + bucket_sequence	50	102172	33038	Agrupar reseñas por producto y periodo para reducir dispersión documental y facilitar consultas analíticas.

Figura 3. MongoDB: efecto del Bucket Pattern sobre reseñas.

6. Optimización MongoDB

La optimización MongoDB se trabajó sobre consultas críticas Q01-Q07 y se validó con explain("executionStats"). Se aplicaron índices compuestos, parciales y de texto, además de un pipeline de agregación con filtrado temprano, proyección selectiva, agrupación y ordenamiento.

Tabla 14. Consultas críticas MongoDB Q01-Q07.

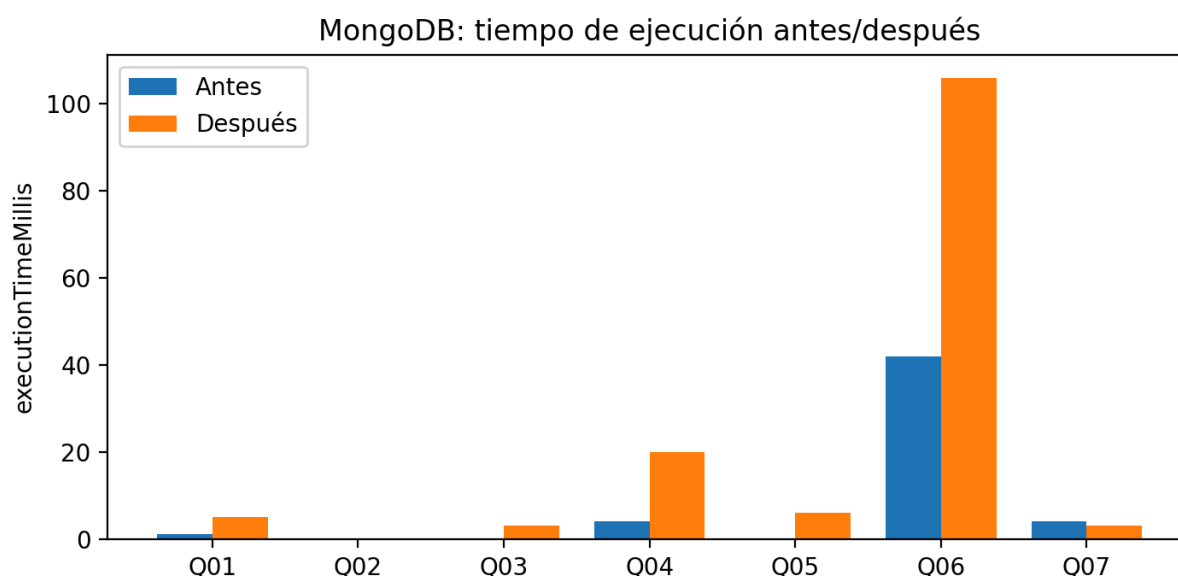
Consulta	Colección	Operación	Propósito
Q01	product_catalog	find	Catálogo por categoría con ordenamiento comercial.
Q02	product_catalog	find	Filtro por categoría y rango de precio.
Q03	product_catalog	find	Consulta por vendedor y atributos resumidos.
Q04	product_catalog	find/text	Búsqueda textual del catálogo.
Q05	product_reviews	find	Reseñas por producto y fecha.
Q06	product_reviews	find	Reseñas críticas usando índice parcial por score.
Q07	product_catalog	aggregate	Agregación analítica por categoría.

Tabla 15. Índices MongoDB implementados o reutilizados.

Índice	Colección	Tipo	Campos	Justificación
idx_catalog_category_price	product_catalog	Compuesto	category + price_summary	Filtros ESR por igualdad/categoría y rango de precio.
idx_catalog_category_sales	product_catalog	Compuesto	category + sales_summary	Ordenamientos y ranking comercial por categoría.
idx_text_product_search	product_catalog	Texto	name + search_keywords	Búsqueda full-text del catálogo.
idx_reviews_product_date	product_reviews	Compuesto	product_id + review_date	Consulta de reseñas por producto y tiempo.
idx_reviews_low_score_partial	product_reviews	Parcial	review_score bajo	Subconjunto de reseñas críticas.
idx_review_buckets_product_period_sequence	product_review_buckets	Compuesto	product_id + bucket_period + bucket_sequence	Acceso eficiente a buckets de reseñas.

Tabla 16. MongoDB: comparación antes/después por consulta.

Consulta	Tiempo ms (antes -> después)	Docs examinados (antes -> después)	Plan (antes -> después)	Lectura técnica
Q01	1 -> 5	2 -> 2	IXSCAN -> IXSCAN	No se observa mejora significativa o la consulta ya estaba optimizada.
Q02	0 -> 0	1 -> 1	IXSCAN -> IXSCAN	No se observa mejora significativa o la consulta ya estaba optimizada.
Q03	0 -> 3	1 -> 1	IXSCAN -> IXSCAN	No se observa mejora significativa o la consulta ya estaba optimizada.
Q04	4 -> 20	1000 -> 1000	COLLSCAN -> IXSCAN	Cambió de escaneo completo a uso de índice.
Q05	0 -> 6	1 -> 1	IXSCAN -> IXSCAN	No se observa mejora significativa o la consulta ya estaba optimizada.
Q06	42 -> 106	15275 -> 15275	IXSCAN -> IXSCAN	No se observa mejora significativa o la consulta ya estaba optimizada.
Q07	4 -> 3	0 -> 0	IXSCAN -> IXSCAN	Redujo tiempo de ejecución.

Figura 4. MongoDB: executionTimeMillis antes/después.**Tabla 17. Pipeline de agregación optimizado en MongoDB.**

Orden	Stage	Propósito
1	\$match	Filtrar temprano por categoría para reducir documentos procesados.
2	\$project	Reducir campos antes de hacer lookup y agrupaciones.
3	\$lookup	Unir productos del catálogo con sus reseñas.
4	\$unwind	Descomponer el arreglo de reseñas para calcular métricas.
5	\$group	Agrupar por categoría y calcular métricas agregadas.
6	\$project	Formatear la salida final del análisis.
7	\$sort	Ordenar categorías por desempeño comercial.

Tabla 18. Comparación base vs optimizada del pipeline MongoDB.

Versión	Tiempo ms	Docs examinados	Keys examinadas	Resultados	Índices usados
BASE	1	3	3	2	Índices de categoría/precio/ventas/rating
OPTIMIZED	1	3	3	2	Índices de categoría/precio/ventas/rating

El pipeline base y el optimizado reportaron 1 ms y 3 documentos examinados. Este resultado indica que el patrón de consulta ya era eficiente con los índices disponibles; la mejora principal fue estructural y de buenas prácticas: filtrar temprano, proyectar campos necesarios y evitar procesar datos innecesarios.

7. Evidencias cuantitativas consolidadas

La siguiente síntesis consolida los hallazgos principales de rendimiento. Para PostgreSQL se integran las métricas del notebook U4 Etapa 2 y el manifiesto incluido en el ZIP; para MongoDB se usan los CSV de explain antes/después y evidencias de Bucket Pattern.

Tabla 19. Evidencias cuantitativas consolidadas.

Evidencia	Medición	Resultado	Interpretación
PostgreSQL Q08	1143,251 ms -> 225,299 ms	80,29% reducción	Mayor mejora relativa documentada.
PostgreSQL Q09	414,951 ms -> 105,594 ms	74,55% reducción	Reescritura evita cálculo repetido de disponibilidad.
PostgreSQL Q02	909,747 ms -> 302,991 ms	66,70% reducción	Índice compuesto por cliente y fecha favorece historial.
PostgreSQL Q04	Shared read 1391 -> 438	68,51% reducción de lecturas	BRIN + reescritura temporal mejora consulta analítica.
MongoDB Q04	COLLSCAN -> IXSCAN	Cambio de plan	Índice de texto habilita búsqueda del catálogo con índice.
MongoDB Q07	4 ms -> 3 ms	25% reducción	Agregación por categoría reduce tiempo observado.
MongoDB Bucket Pattern	102.172 -> 33.038 documentos	Agrupación documental	Reduce dispersión de reseñas por producto/periodo.
MongoDB JSON Schema	Documento inválido rechazado	Resultado esperado	Validador product_catalog opera con action=error.

8. Sincronización entre PostgreSQL y MongoDB

La sincronización se plantea como flujo batch/ETL académico. PostgreSQL conserva la fuente de verdad transaccional; MongoDB almacena documentos derivados y enriquecidos para catálogo, reseñas y análisis. El vínculo entre motores se mantiene mediante identificadores compartidos como product_id, order_id, seller_id y, cuando aplica, customer_id.

Tabla 20. Relación de datos entre PostgreSQL y MongoDB.

Dominio	Origen PostgreSQL	Destino MongoDB	Clave de integración	Uso
Productos	core.products / core.product_categories	product_catalog	product_id, category	Catálogo enriquecido y búsqueda flexible.
Vendedores	core.sellers	seller_summary dentro de product_catalog	seller_id	Extended Reference para evitar joins frecuentes.
Pedidos y ventas	core.orders / core.order_items	sales_summary dentro de product_catalog	product_id, order_id	Computed Pattern para métricas precalculadas.
Reseñas	core.reviews_ref / product_reviews	product_reviews y product_review_buckets	product_id, review_id	Referencing y Bucket Pattern para análisis de reseñas.

La consistencia se interpreta como eventual: los cambios transaccionales se reflejan en MongoDB mediante procesos de actualización programados o notebooks de carga. Para producción se recomienda evolucionar hacia CDC, colas de eventos o jobs automatizados con control de idempotencia, pero esto no hace parte del alcance exigido para el prototipo evaluativo.

9. Sharding y replica sets

El diseño de sharding y replica sets se documenta de forma teórica, sin habilitar sharding real sobre el clúster Atlas académico. Esta decisión evita riesgo operativo y es coherente con las limitaciones de free tier, manteniendo la capacidad de justificar escalabilidad futura.

Tabla 21. Shard keys propuestas para MongoDB.

Colección	Shard key propuesta	Justificación	Trade-off
product_catalog	category + hashed product_id	Permite consultar por categoría y distribuir por product_id para evitar concentración en categorías populares.	Evitar category como única shard key por baja cardinalidad relativa.
product_reviews	hashed product_id	Distribuye reseñas por producto y evita hotspot por crecimiento de reseñas.	Revisar consultas que requieran rango temporal global.
product_review_buckets	product_id + bucket_period + bucket_sequence	Consulta dirigida por producto/periodo; consistente con Bucket Pattern.	Balancear tamaño de bucket si cambia volumen.

Tabla 22. Estrategia teórica de replica set, read preferences y write concerns.

Operación	Read preference / destino	Read/Write concern	Criterio
Escrituras críticas	primary	writeConcern majority	Garantiza confirmación en mayoría antes de reportar éxito.
Lecturas de catálogo	primaryPreferred	readConcern local/majority según criticidad	Prioriza actualidad, tolerando fallback cuando aplique.
Analítica y reportes	secondaryPreferred	readConcern local	Descarga lecturas no críticas a secundarios.
Cargas batch	primary	majority + wtimeout	Controla durabilidad y evita bloqueos indefinidos.

10. Conclusiones

- La implementación final mantiene una arquitectura coherente: PostgreSQL/Supabase como núcleo transaccional y MongoDB Atlas como complemento documental y analítico.
- PostgreSQL quedó validado con esquema core, tablas principales, constraints, extensiones, índices, particionamiento y evidencia EXPLAIN antes/después soportada por la Unidad 4.
- MongoDB quedó documentado con colecciones orientadas a catálogo y reseñas, JSON Schema, Bucket Pattern, índices optimizados, consultas Q01-Q07 y pipeline de agregación.
- Las evidencias cuantitativas muestran mejoras importantes en PostgreSQL y cambios de plan relevantes en MongoDB, aunque algunas consultas ya estaban optimizadas o no mejoraron por factores de selectividad/caché.
- El repositorio queda listo para evaluación y demostración, con documentación, scripts, notebooks, evidencias y checklist de video.

11. Referencias

- Universidad de La Sabana. (2026). Guía de actividades - Unidad 5: Optimización de rendimiento en MongoDB.
- Equipo Ecommify. (2026). Documento técnico de diseño: Diseño conceptual y lógico de la base de datos del proyecto Ecommify.
- Equipo Ecommify. (2026). Optimización de rendimiento en PostgreSQL - Etapa 2.
- Equipo Ecommify. (2026). Optimización de rendimiento en MongoDB - Etapa 1.
- PostgreSQL Global Development Group. (2025). PostgreSQL Documentation: Performance tips. <https://www.postgresql.org/docs/current/performance-tips.html>
- MongoDB Inc. (2025). MongoDB Manual: Analyzing MongoDB performance. <https://www.mongodb.com/docs/manual/administration/analyzing-mongodb-performance/>
- MongoDB Inc. (2025). Schema design patterns. MongoDB Documentation. <https://www.mongodb.com/docs/manual/data-modeling/design-patterns/>
- Bradshaw, S., Brazil, E., & Chodorow, K. (2019). MongoDB: The Definitive Guide: Powerful and Scalable Data Storage (3rd ed.). O'Reilly Media.
- Borucki, A. (2025). MongoDB 8.0 in Action: Building on the Atlas Data Platform (3rd ed.). Manning Publications.
- Schönig, H. (2020). Mastering PostgreSQL 13: Build, administer, and maintain database applications efficiently with PostgreSQL 13 (4th ed.). Packt Publishing.

- Riggs, S., Ciolli, G., & Meesala, S. K. (2019). PostgreSQL 11 Administration Cookbook. Packt Publishing.

Anexos

Los anexos resumen la trazabilidad contra rúbrica, el inventario del repositorio y las evidencias complementarias usadas para construir el documento final.

Anexo A. Evidencias PostgreSQL

Tabla A1. Checklist de validación PostgreSQL.

Validación	Estado	Evidencia	Notas
schema_core_exists	OK	Consulta sobre information_schema.schemata	El esquema core existe en Supabase PostgreSQL.
core_tables_exist	OK	Consulta sobre pg_tables	Las tablas principales del modelo Ecommify existen en core.
row_estimates_available	OK	Consulta sobre pg_class y pg_namespace	Las tablas tienen conteos estimados disponibles.
extensions_checked	OK	Consulta sobre pg_extension	Se validaron extensiones relevantes para el modelo.
constraints_checked	OK	Consulta sobre information_schema.table_constraints	Se validaron primary keys foreign keys check constraints y otros constraints.
orders_partitioning_checked	OK	Consulta sobre pg_partitioned_table	La tabla core.orders aparece particionada por purchase_date.
indexes_inventory_checked	OK	Consulta sobre pg_indexes	Se validó inventario de índices del esquema core.
explain_control_query_executed	OK	EXPLAIN ANALYZE BUFFERS	La consulta crítica de control ejecutó correctamente.

Tabla A2. Evidencias documentadas de EXPLAIN PostgreSQL.

Evidencia	Estado	Descripción	Referencia fuente
baseline_explain	documented	Medición base de Q01-Q09 con EXPLAIN ANALYZE BUFFERS	U4 Etapa 2 Hito 4
post_optimization_explain	documented	Medición posterior después de optimizaciones e índices	U4 Etapa 2 Hito 8
before_after_comparison	documented	Comparación antes/después con reducción de tiempo y bloques leídos cuando aplica	U4 Etapa 2 Hito 8
specialized_indexes	documented	Índices B-tree parcial GIN pg_trgm y BRIN documentados	U4 Etapa 2 Hito 7
partitioning_validation	documented	Validación de particionamiento declarativo de core.orders por purchase_date	U4 Etapa 2 Hito 9

Anexo B. Evidencias MongoDB

Tabla B1. Explain de consulta sobre product_review_buckets.

Consulta	Colección	Operación	Winning stage	Índice	Docs examinados	Keys examinadas	Tiempo ms	Resultados
BKT01	product_review_buckets	find	FETCH	idx_review_buckets_product_period_sequence	1	1	0	1

Tabla B2. Stages del pipeline optimizado.

Orden	Stage	Propósito
1	\$match	Filtrar temprano por categoría para reducir documentos procesados.
2	\$project	Reducir campos antes de hacer lookup y agrupaciones.
3	\$lookup	Unir productos del catálogo con sus reseñas.
4	\$unwind	Descomponer el arreglo de reseñas para calcular métricas.
5	\$group	Agrupar por categoría y calcular métricas agregadas.
6	\$project	Formatear la salida final del análisis.
7	\$sort	Ordenar categorías por desempeño comercial.